H

HELPYARD

Graphical Workspace Upgrade

UI/UX redesign • Interactive analytics • MCP integrations • Workbench & instructions • Motion and 3D • Safe implementation roadmap

PROJECT UPGRADE IMPLEMENTATION PLAN

Prepared from the 14-page “Project Report Number 1” assessment. This plan preserves the existing local-first architecture and separates verified capabilities from proposed enhancements.

DESIGN DIRECTION

A polished developer command center: clear hierarchy, restrained motion, meaningful charts, responsive navigation, and explicit trust/safety states.

BASELINE: local HTML dashboard + Windows PowerShell HTTP bridge + optional Ollama

Status in source report: verified prototype assessment; public release not yet approved.

1. Executive direction

The report describes Helpyard as a single-page HTML/CSS/JavaScript dashboard paired with a Windows PowerShell HTTP bridge, with optional local Ollama inference and optional direct cloud AI. The redesign should improve clarity and workflow efficiency without treating visual polish as a substitute for server-side safety.

Area	Current report baseline	Upgrade target
Overview	Live host metrics and history, but some errors are silently caught.	A calm command-center layout with readable trends, freshness timestamps, clear loading/error/empty states, and drill-down links.
Navigation	12 sections; sidebar moves above content at narrow widths.	Grouped, searchable navigation; compact/collapsible rail; keyboard shortcuts; mobile drawer; persistent page context.
MCP	Cards are mostly descriptive; lifecycle API routes exist in bridge.	Real server inventory and lifecycle controls only after executable allowlisting, validation, policy checks, and audit logging.
Workbench / instructions	Context label and system prompt are browser-state features; action cards are predefined.	Instruction profiles, project context, task plans, approval gates, progress timeline, and reusable workflows.
Security	Static posture labels; no persistent event audit log.	Evidence-backed status cards, recent events, policy explanation, and honest “not checked” states.
Visual layer	Dark navy with cyan/violet accents; charts/cards; limited light-theme consistency.	Design system, accessible charts, subtle animation, optional 3D hero/scene, reduced-motion fallback.

Non-negotiable release principle

Keep the real workstation bridge loopback-only. Never expose the machine-control bridge to the public internet or place it behind a public tunnel. A public demo must use a separate mock backend with synthetic data and no workstation access.

Recommended order: safety and data correctness → information architecture → charts and visual hierarchy → workflow integrations → motion/3D polish.

2. Design goals and experience principles

- **Make the overview answer four questions quickly:** Is the bridge connected? What needs attention? What is running? What can I safely do next?
- **Separate facts from suggestions:** live values, cached/stale values, static guidance, AI-generated suggestions, and unverified integrations must look different.
- **Use charts only when they help decisions:** trends, capacity, change over time, and distribution; avoid decorative gauges that repeat the same value.
- **Progressive disclosure:** summary first, detail on click, raw output and technical metadata in expandable panels.
- **Safe-by-default interactions:** preview impact, name the target, require explicit approval for consequential actions, and show completion/failure evidence.
- **Accessible motion:** short transitions, no perpetual motion in data panels, respect reduced-motion settings, and preserve keyboard focus.

3. Proposed navigation and page architecture

Retain the current product scope but group it into coherent work areas. Do not add a navigation item until its function is implemented or clearly labelled as planned.

Group	Screens / sections	Key improvement
Command Center	Overview, Activity / Audit	Overview becomes a decision surface; Activity is backed by real events, not hard-coded labels.
Workspace	Projects, Project Explorer, Workbench	Approved workspace picker, safe tree view, project metadata, task plan and reviewable edits.
Runtime	Processes, Services, Terminal / Tasks	Filters, target details, risk badges, scoped named actions; arbitrary shell disabled by default.
AI & Integrations	Local AI, Cloud AI, MCP, Skills	Connection health, model/provider status, per-integration settings, explicit privacy labels.
Security & Settings	Security Center, Instructions / Profiles, Settings	Evidence-based checks, editable instruction profiles, connection settings and data controls.

Overview layout blueprint

Zone	Recommended contents	Visual treatment
Top bar	Page title, bridge status, host label, last refresh, refresh and global search.	Compact, sticky; connection state includes text and icon, not color alone.
Attention strip	Action required, stale metrics, failed integrations, pending approvals.	Only appears when needed; dismissible notices remain in Activity.
Metric row	CPU, memory, storage, uptime/process count (choose 4–5 max).	Large value + unit + small trend; tooltips show collection time/source.
Analytics	CPU/RAM time series; storage capacity by drive.	Consistent axes and units; selectable time range only if backend supports it.
Workspace & AI	Recent approved projects, Ollama health/model, MCP server health.	Real status chips and empty states; no fake “connected” states.
Activity	Recent actions, approvals, errors, and task runs.	Chronological timeline with target, outcome, duration, and safe redaction.
Quick actions	Open project, refresh health, start a predefined task.	Small set of frequent actions; risk and permission shown before execution.

Visual hierarchy sketch (concept only)

HELPYARD / Overview ● Connected
Local workstation • refreshed just now

ATTENTION One integration offline · 2 actions awaiting approval · metrics current

CPU 21% **MEMORY** 87% **STORAGE** 69% **UPTIME** 11.5 h

CPU / MEMORY HISTORY [line chart] **DISK CAPACITY** [bar chart]

PROJECTS recent approved workspaces **AI & MCP** live integration status

RECENT ACTIVITY action · target · status · time · inspect details

This is a structural wireframe, not a screenshot or claim about current live values.

4. Charts, data visualization, and dashboard intelligence

The report confirms overview metrics and CPU/RAM history are rendered, but it also notes that some loader failures are silently caught. First make the data trustworthy; then upgrade chart fidelity.

Visualization	Purpose / interaction	Data contract and guardrail
CPU & memory time series	Show trend, hover details, selectable range if supported.	Timestamp + numeric value + unit; show gaps and stale status instead of connecting missing data as if real.
Disk capacity bars	Compare used/free capacity by drive; click for details.	Drive ID, used bytes, total bytes, collection timestamp; handle zero/unknown total.
Process distribution	Optional top-process memory/CPU ranking; filter and sort.	Only add CPU share if the API actually provides it; show process count and sampling time.
Service state summary	Show running/stopped/other counts and drill into table.	Use returned service states; never infer service health from labels alone.
Integration health map	Local bridge, Ollama, MCP servers and provider configuration.	Each status has source, checked-at time, and last error; “not configured” differs from “offline.”
Activity timeline	Trace action → approval → execution → result.	Requires persistent, redacted audit events; do not manufacture history from current state.

Chart design rules

- Use a restrained palette: cyan for active data, violet for AI/workflow, amber for warning, red for dangerous/failure, neutral gray for unknown.
- Always label axes, units, time window and sample interval. Avoid 3D chart bars, perspective pie charts, and unnecessary gradients that distort comparisons.
- Use skeleton loading only briefly; provide empty, offline, stale, partial-data, and retry states.
- Tooltips must work with pointer and keyboard; color must not be the only status cue.
- Do not display percentages or uptime as illustrative placeholders when real data is unavailable. Use “—” and explain why.

5. Design system and graphical polish

Token / component	Recommendation
Color	Base #0B1220 / #14213A, surface #18263D, text #F5F8FC, muted #9BAAC0, cyan #00C2D7, violet #8B5CF6. Define semantic success/warning/error/unknown tokens.
Typography	Use one clean UI family (e.g., system UI/Inter where available); 12–14 px body minimum on dense screens; clear heading scale.
Spacing / radius	4 px spacing scale; consistent 10–16 px card radius; avoid every component looking like a separate floating panel.
Cards	Use a clear title, value/summary, context, and action; avoid thick borders and multiple nested cards.
Navigation	Active state uses icon + label + accent marker; groups collapse; focus and hover states are distinct.
Tables	Sticky header where useful, column sorting, search/filter, pagination or virtualization for large lists, responsive overflow.
Light theme	Define separate surface/text/border/shadow tokens; audit all charts, forms, badges, menus and dialogs rather than only changing body background.
Motion	Use 120–220 ms transitions for hover/panel entry; use spring-like movement sparingly; no animation that hides a state change.

6. New and upgraded sections

These are proposed additions, not capabilities claimed by the report to exist today.

Section	Features to implement	Dependencies / safeguards
MCP Manager	Configured server list; status/health; command and argument preview; start/stop/restart where supported; logs; environment configuration; test connection; per-server detail drawer.	Bridge already has status/start/stop route patterns, but UI wiring is incomplete. Allowlist executable paths; validate args/env; avoid shell interpolation; restrict working directories; redact secrets; enforce central policy and audit.
Instructions & Modes	Named instruction profiles (e.g., Developer, Research, Security Review, Documentation); mode description; allowed tools; context scope; approval level; model choice; temperature/settings only where provider supports them; preview active instructions.	A mode is guidance, not authorization. Enforce tool permissions on the bridge regardless of prompt text. Do not silently give profiles more system access.
Workbench / Task Planner	Goal, scope, checklist, step status, evidence/output links, blockers, approval gates, retry and cancel, final summary.	Do not let the plan directly execute arbitrary commands. Each step maps to a named, validated operation.
Project Explorer	User-approved folder picker; tree, language/manifest detection, Git branch/status, search, exclude rules, open folder, project-specific context.	No broad automatic crawling by default. Exclude secrets/build output; show scanned paths; allow clear/remove context.
Security Center	Real checks, severity, evidence, last run, remediation guidance, audit trail, exportable report.	Until implemented, mark each item “Guidance” or “Not checked.” Do not claim compliance or scan pass based on static labels.
Activity & Audit	Filter by action, target, outcome, actor/context; inspect details; export redacted events.	Persistent event schema, retention controls, secret redaction; avoid storing full command bodies unless essential.
AI Studio	Local model status, model selection, chat streaming/cancel, prompt profiles, context sources, token/latency indicators if available.	Local/cloud destinations clearly labelled; cloud API keys must not be stored in plain localStorage; show provider data disclosure.
Preferences	Theme, reduced motion, density, chart range, default landing page, keyboard shortcuts, privacy/history controls.	Settings should be validated, versioned and recoverable; corrupt local state must not break app startup.

Instructions / modes interaction model

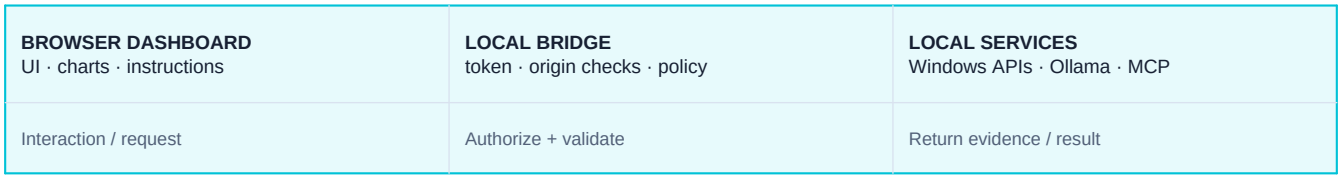
- Mode selector displays name, purpose, current model/provider, allowed tool categories, and whether approval is required.
- Instruction editor supports version history, reset-to-default, validation, preview, and explicit Save/Activate actions.
- Project context is opt-in and scoped to approved folders. Show exactly which files/snippets are included in a request where feasible.
- A “Plan first” mode produces a task checklist without running commands; “Ask before changes” requires per-action approval; “Read-only” blocks mutation endpoints server-side.
- Display a persistent banner when cloud AI is active and show what data leaves the machine.

7. Motion, animation, and 3D effects

Use motion to explain hierarchy and state—not to decorate every panel. The workstation dashboard is operational software, so readability, responsiveness and reduced distraction matter more than spectacle.

Effect	Where it fits	Implementation guidance
Animated metric count-up	First successful load or explicit refresh only.	Animate from prior value to new value briefly; respect reduced motion; do not animate every polling tick.
Chart drawing / transitions	CPU/RAM trend, disk capacity updates.	Use short transitions; maintain stable axes; no dramatic rescaling that makes small changes look large.
Connection pulse	Bridge / Ollama / MCP connection indicator.	Subtle pulse only during connecting/checking; static icon + text for steady states.
Page transitions	Navigation between major areas.	Fade/translate 4–8 px, 120–180 ms; avoid blocking input or resetting scroll unnecessarily.
3D hero / workspace visualization	Optional Overview header or dedicated Workspace scene.	Lightweight CSS 3D or a small WebGL scene; use as progressive enhancement, never as a chart replacement.
Interactive system map	Bridge → Windows APIs / Ollama / MCP servers.	Clickable nodes show real status and data path; use a 2D SVG/canvas default and optional 3D view.
Micro-interactions	Buttons, tabs, drawers, approvals, copy actions.	Clear focus/pressed/disabled states; motion never substitutes for success/error text.

3D system map concept



Use visible directional arrows in the implemented diagram. Do not imply the browser itself enforces authorization; the bridge is the policy enforcement point.

Performance and accessibility constraints

- Provide a static, useful dashboard if WebGL is unavailable or disabled; lazy-load 3D only when visible.
- Aim for no meaningful delay to initial interaction; avoid large animation libraries if CSS/SVG can achieve the same result.
- Honor prefers-reduced-motion and add a user preference. Remove parallax, count-ups and looping pulses when enabled.
- Support keyboard-only use, visible focus, screen-reader labels, contrast, zoom, and small screens.
- Never use motion alone to communicate connection, risk, completion, or failure.

8. Technical implementation architecture

Preserve the existing single-file frontend initially to reduce migration risk. If the interface grows substantially, split into modules after behavior is covered by tests; do not mix a large visual rewrite with a bridge-security rewrite in one unreviewable change.

Layer	Implementation tasks
Frontend shell	Create CSS variables/tokens, shared components, route/section registry, responsive sidebar, toast/alert system, dialog/drawer, chart wrapper, theme and reduced-motion support.
Data state	Centralize API calls and loading/error states; attach timestamps and request IDs; use timeouts and bounded polling; prevent stale responses from overwriting newer ones.
Bridge policy	Central authorization/policy function for every mutating route; named operation registry; strict JSON schemas; target allowlists; role/approval checks; execution time and output caps.
MCP lifecycle	Accept only configured server IDs mapped to reviewed executable paths and fixed argument templates; never trust a UI-supplied arbitrary executable path.
AI providers	Provider adapter abstraction; destination label; timeout/cancel; redact secrets; no cloud key in plain localStorage; explicit history/context controls.
Audit / observability	Persistent event store with timestamp, operation ID, target, decision, result and duration; redact secrets; export only after review.
Testing	Unit tests for formatting/state; API contract tests; negative authorization tests; browser screenshot/regression tests; manual accessibility checks.

Suggested frontend structure (future refactor)

```
src/
app/ navigation, routing, global state
components/ cards, charts, tables, dialogs, status, toasts
features/ overview, projects, workbench, processes, services,
terminal, local-ai, cloud-ai, mcp, skills, security, settings
services/ bridge-client, ollama-client, provider adapters
styles/ tokens, themes, motion, responsive rules
tests/ unit, API contract, accessibility, browser regression
```

Security gates before connecting the new UI

- Disable unrestricted /api/run by default or replace it with named, parameter-validated operations.
- Enforce authorization, approval, least privilege and audit on the server for process/service/MCP/config mutations; browser confirmations are not a security boundary.
- Add body-size limits, command timeouts, cancellation, output caps, safe shutdown and malformed-input handling.
- Keep bridge on 127.0.0.1; validate Host/Origin and authentication; never place live bridge behind a public tunnel.
- Do not store long-lived secrets unnecessarily in localStorage. Separate cloud credentials from bridge tokens and provide clear/logout controls.
- Use mock data and a separate backend for any public showcase.

9. Phased implementation roadmap

The roadmap is deliberately ordered so a polished interface does not conceal unsafe or inaccurate behavior. Estimate is a planning guide for one developer and assumes access to the source code and a test Windows machine.

Phase	Work packages	Exit / acceptance criteria	Priority
0 · Baseline	Tag current version; capture screenshots; inventory endpoints and UI states; create test data and rollback branch.	Reproducible startup; known current behavior recorded; no baseline regressions.	P0
1 · Secure foundation	Central bridge policy; restrict /api/run; validate service/process/MCP/config requests; bounds, audit, error reporting.	Negative tests prove rejected requests do not change system state; audit records outcomes.	P0
2 · Data correctness	Shared API client; timestamps; stale/error/empty states; retries; consistent loading skeletons; remove silent failures.	Offline, malformed response, timeout and partial-data states are visible and tested.	P0
3 · Design system	Tokens, typography, responsive shell, sidebar/search, theme consistency, focus, reduced motion.	Desktop and narrow layouts usable; keyboard and contrast review passes.	P1
4 · Overview redesign	Metric hierarchy, CPU/RAM charts, disk bars, integration health, recent activity and quick actions.	Every metric has source/time; no placeholder presented as live; charts verified against API values.	P1
5 · Workbench + modes	Instruction profiles, plan-first workflow, approval steps, task history, project-scoped context.	Mode cannot bypass bridge policy; saved instruction changes are reversible and validated.	P1
6 · MCP Manager	Configured server registry, health checks, lifecycle UI, logs, validation and safe error display.	Only allowlisted configured server IDs can start/stop; secrets are redacted; audit entries exist.	P1/P2
7 · Project intelligence	Approved roots, project tree, manifest/language metadata, safe search/excludes.	Scope visible; excluded secrets/build folders tested; user can clear project context.	P2
8 · Motion + optional 3D	Micro-interactions, chart transitions, optional system map/3D scene, fallback and reduced motion.	Performance/accessibility checks pass; no core function depends on 3D.	P2
9 · Qualification / demo	Browser matrix, regression automation, installation docs, mock public demo, release notes and rollback.	P0/P1 gates complete; public demo has no bridge, credentials, or live workstation data.	P3

Suggested delivery slices

Slice	Deliverable	Planning range
A	Safety + accurate states	3–6 developer days
B	Visual system + overview	3–5 developer days
C	Workbench/instructions + audit UI	4–7 developer days
D	MCP manager + project explorer	4–8 developer days
E	Motion/3D + accessibility/performance	2–4 developer days
F	Regression + browser validation + documentation	3–6 developer days

These ranges are estimates, not commitments. Actual effort depends on repository state, test coverage, backend policy changes, and the complexity of MCP server configuration.

10. Verification plan and acceptance checklist

Run checks against a dedicated test environment first. Tests that mutate services, processes, shutdown state, or configuration should use safe fixtures and explicit operator approval.

Area	Required tests
Authentication / origin	Valid token succeeds; missing/invalid token fails; hostile origin fails; wrong host fails; expired/restarted token requires reconnect.
Input validation	Malformed JSON, unknown fields, invalid IDs, oversized body, unexpected types, command injection strings, missing required fields.
Authorization	Read-only mode blocks mutations; insufficient privilege denied; approval-required operation cannot run without server-verified approval.
Resource bounds	Command timeout, output cap, cancellation, repeated requests, server stop/restart, port conflict and graceful cleanup.
UI states	Loading, no data, disconnected, stale, partial, error, retry success, repeated refresh, slow responses and browser storage unavailable.
Charts	Known fixture values match labels/units; missing samples create gaps; responsive resize; keyboard tooltip; color-independent labels.
MCP	Unknown server ID rejected; executable path not accepted from arbitrary UI input; args validated; stop only tracked process; logs redact secrets.
Instructions / modes	Save/reload/version/restore; prompt preview; cloud destination warning; mode cannot grant new server privileges.
Accessibility / visual	Keyboard traversal, visible focus, contrast, reduced motion, 200% zoom, narrow viewport, table overflow, screen-reader labels.
Release separation	Public demo network inspection proves no calls to live bridge, Ollama, local files, process APIs, or real credentials.

Definition of done for the graphical upgrade

- The overview is understandable within 10 seconds and has no misleading or unlabelled values.
- All integration and security states are backed by real checks or explicitly labelled as guidance/not checked.
- Every high-impact operation shows target, risk, approval requirement and result; enforcement occurs in the bridge.
- MCP and instruction/mode screens are fully wired or explicitly labelled “planned”—no fake controls.
- The interface works with keyboard, reduced motion, light/dark themes and narrow screens.
- The 3D layer is optional, lightweight and never blocks operational use.
- A public-facing build is isolated from the live workstation bridge.

11. Deliverables, priorities, and developer handoff

Recommended implementation deliverables

Deliverable	Contents
UI/UX specification	Navigation map, overview wireframe, component states, responsive rules, chart definitions, theme tokens.
Design system	Colors, typography, spacing, cards, tables, forms, badges, dialogs, toasts, focus and motion tokens.
API/policy contract	Endpoint schemas, error formats, operation registry, authorization/approval rules, timeouts, output caps.
MCP configuration model	Stable server IDs, reviewed executable path, argument templates, environment secret references, working directory, health state.
Instruction/mode schema	Profile ID, purpose, prompt text, allowed tool categories, context scope, approval policy reference, version.
Audit event schema	Timestamp, operation ID, target, request source, policy decision, result, duration and redaction status.
Test pack	Unit tests, bridge contract tests, hostile-input tests, UI screenshots, accessibility checklist and release evidence.
Release package	Startup/stop guide, recovery steps, privacy note, known limitations, version notes, rollback and isolated mock demo.

Developer instructions

- Start by inspecting the existing pc-control-dashboard.html, bridge.ps1 and README.md; do not rewrite based only on the report.
- Create a baseline branch and record current working behavior before changing layout or endpoints.
- Implement P0 safety and data-state work first. Treat every route listed in the report as untrusted input until its policy is verified.
- Build the overview redesign against real API responses and deterministic fixtures; label any mock/demo mode prominently.
- Wire MCP lifecycle controls only through a reviewed server registry and bridge-side policy. Never pass arbitrary executable paths from the browser.
- Implement instruction profiles as workflow configuration, not as an authorization system.
- Add animation after layout and data states are stable; keep 3D optional and test the fallback.
- Record screenshots, browser versions, test commands, pass/fail evidence and unresolved limitations in each release.

12. Source alignment and assumptions

This plan is based on the uploaded “Project Report Number 1” (14 pages, prepared 17 October 2026). Source-derived baseline facts include: single-page HTML/CSS/JS dashboard; Windows PowerShell HTTP bridge bound to loopback; optional Ollama; 12 navigation sections; verified local metrics/process/service/project views and non-streaming local chat in one Windows Chromium-based environment; MCP UI incomplete despite bridge route patterns; static security indicators; and public release not approved.

Proposed additions in this plan—not verified current capabilities—include the MCP Manager UI, instruction/mode profiles, activity/audit UI, project explorer, improved chart interactions, optional 3D system map, and expanded workflow planning.

The report states Firefox was unavailable in the test environment and that a full browser matrix, comprehensive automated tests, adversarial security review, mutation-route tests, and public staging deployment were not performed. These remain validation work, not presumed passes.

Final recommendation

Treat the upgrade as a product-quality and safety project, not a visual-only redesign. Deliver a trustworthy overview first; then add Workbench modes and a real MCP manager; add motion and 3D last. The result should feel modern and graphical while remaining fast, accessible, honest about data, and safe for a local machine-control dashboard.